

Swagger-шпаргалка для защиты

Индивидуальный проект РБД · Ермаков Л. · М09-КИИ26

Как показывать работу стенда в Swagger UI: эндпоинты, сценарии, SQL, отказы

0. Как пользоваться этим документом на защите

- Откройте Swagger: <http://127.0.0.1:8000/docs>
- Держите рядом этот PDF: справа — что нажать, ниже — какой SQL уйдёт в БД.
- После каждого шага можете открыть psql и показать строки в таблицах.
- Красные блоки §4 — это намеренные отказы (не баг): автомат статусов, контур, нехватка остатка, отказ primary.
- Параметр {region} в URL — всегда WEST или EAST (регистр не важен).

0.1. Маршрутизация (скажите преподавателю вслух)

Тип запроса	Узел	Пример
GET (чтение)	сначала replica, иначе primary	GET /api/WEST/stock
POST/PUT/DELETE (запись)	только primary региона	POST /api/WEST/orders
Смена статуса	primary + журнал shipment_events	POST .../orders/{id}/status
Чужой регион в URL	отказ 403 (критерий контура)	EAST + id заявки WEST

В каждом успешном ответе смотрите блок routing: region, role (primary|replica), host.

1. Как зайти в каждую базу (терминал)

Учётка учебная: пользователь/пароль/БД = warehouse. Два способа: через psql с хоста или через docker exec.

1.1. С хоста (порты Docker)

```
# WEST primary (запись WEST)
PGPASSWORD=warehouse psql -h 127.0.0.1 -p 15432 -U warehouse -d warehouse

# WEST replica (чтение WEST, только SELECT)
PGPASSWORD=warehouse psql -h 127.0.0.1 -p 15433 -U warehouse -d warehouse

# EAST primary (запись EAST)
PGPASSWORD=warehouse psql -h 127.0.0.1 -p 15434 -U warehouse -d warehouse

# EAST replica (чтение EAST)
PGPASSWORD=warehouse psql -h 127.0.0.1 -p 15435 -U warehouse -d warehouse
```

1.2. Через docker exec (если psql не установлен)

```
docker exec -it rdb-west-primary psql -U warehouse -d warehouse
docker exec -it rdb-west-replica psql -U warehouse -d warehouse
docker exec -it rdb-east-primary psql -U warehouse -d warehouse
docker exec -it rdb-east-replica psql -U warehouse -d warehouse
```

1.3. Полезные SELECT «что смотреть после Swagger»

```
-- заявки и статусы
SELECT id, warehouse_id, region_code, status, updated_at
FROM shipment_orders ORDER BY id;

-- журнал переходов
SELECT id, order_id, from_status, to_status, note, created_at
FROM shipment_events ORDER BY id;

-- остатки (available / reserved)
SELECT sb.id, w.code, p.sku, sb.qty_available, sb.qty_reserved
FROM stock_balances sb
JOIN warehouses w ON w.id = sb.warehouse_id
JOIN products p ON p.id = sb.product_id
ORDER BY sb.id;

-- реплика ли я? (на replica = true, на primary = false)
SELECT pg_is_in_recovery();
```

Узел	Контейнер	Порт хоста	Роль
WEST primary	rdb-west-primary	15432	запись WEST
WEST replica	rdb-west-replica	15433	чтение WEST
EAST primary	rdb-east-primary	15434	запись EAST
EAST replica	rdb-east-replica	15435	чтение EAST

2. Каталог эндпоинтов Swagger (каждый запрос)

Нижне — все операции клиента. В Swagger: найдите метод → Try it out → подставьте region / id / JSON → Execute. Ответ смотрите в Response body.

2.1. Служебное

GET /health

Что делает: Проверка всех 4 узлов: WEST/EAST × primary/replica. Не пишет в БД бизнес-данные.

Куда идёт: по очереди все DSN; на каждом SELECT pg_is_in_recovery()

Смотрите nodes[].ok и detail=in_recovery=true/false.

Пример в Swagger (Try it out → Execute)

просто Execute (параметров нет)

SQL на фрагменте (то, что реально уходит в PostgreSQL)

```
SELECT pg_is_in_recovery(); -- на каждом узле
```

2.2. Склады (warehouses)

GET /api/{region}/warehouses

Что делает: Список складов региона (чтение).

Куда идёт: чтение → replica региона

Пример в Swagger (Try it out → Execute)

region = WEST

SQL на фрагменте (то, что реально уходит в PostgreSQL)

```
SELECT w.id, w.code, w.name, r.code AS region_code
FROM warehouses w
JOIN regions r ON r.id = w.region_id
WHERE r.code = 'WEST'
ORDER BY w.id;
```

POST /api/{region}/warehouses

Что делает: Создать склад на primary региона.

Куда идёт: запись → primary

Пример в Swagger (Try it out → Execute)

region=WEST

Body:

```
{
  "code": "WH-WEST-DEMO",
  "name": "Склад Запад DEMO"
}
```

SQL на фрагменте (то, что реально уходит в PostgreSQL)

```
INSERT INTO warehouses (region_id, code, name)
VALUES (<id региона WEST>, 'WH-WEST-DEMO', 'Склад Запад DEMO')
RETURNING id, region_id, code, name;
```

PUT /api/{region}/warehouses/{warehouse_id}

Что делает: Обновить code и/или name склада.

Куда идёт: запись → primary; сначала проверка, что склад этого региона

Пример в Swagger (Try it out → Execute)

region=WEST, warehouse_id=<id>

Body:

```
{
  "name": "Склад Запад DEMO (upd)"
}
```

SQL на фрагменте (то, что реально уходит в PostgreSQL)

```
UPDATE warehouses
SET code = COALESCE(NULL, code),
    name = COALESCE('Склад Запад DEMO (upd)', name)
WHERE id = <id>
RETURNING id, region_id, code, name;
```

DELETE /api/{region}/warehouses/{warehouse_id}

Что делает: Удалить склад (если нет зависимостей / или каскад по схеме).

Куда идёт: запись → primary

Пример в Swagger (Try it out → Execute)

region=WEST, warehouse_id=<id> → Execute

SQL на фрагменте (то, что реально уходит в PostgreSQL)

```
DELETE FROM warehouses WHERE id = <id> RETURNING id;
```

2.3. Товары (products)

Каталог не общий физически: на WEST и EAST свои копии таблицы products. Создали на WEST — на EAST этой строки нет.

GET /api/{region}/products

Что делает: Список товаров фрагмента.

Куда идёт: чтение → replica

Пример в Swagger (Try it out → Execute)

region=WEST

SQL на фрагменте (то, что реально уходит в PostgreSQL)

```
SELECT id, sku, title FROM products ORDER BY id;
```

POST /api/{region}/products

Что делает: Создать товар на primary региона.

Куда идёт: запись → primary

Пример в Swagger (Try it out → Execute)

region=WEST

Body:

```
{
  "sku": "SKU-100",
  "title": "Кабель 1м"
}
```

SQL на фрагменте (то, что реально уходит в PostgreSQL)

```
INSERT INTO products (sku, title)
VALUES ('SKU-100', 'Кабель 1м')
RETURNING id, sku, title;
```

PUT /api/{region}/products/{product_id}

Что делает: Обновить sku/title.

Куда идёт: запись → primary

Пример в Swagger (Try it out → Execute)

region=WEST, product_id=<id>

Body: {"title": "Кабель 1м (upd)"}

SQL на фрагменте (то, что реально уходит в PostgreSQL)

```
UPDATE products
SET sku = COALESCE(NULL, sku),
    title = COALESCE('Кабель 1м (upd)', title)
WHERE id = <id>
RETURNING id, sku, title;
```

DELETE /api/{region}/products/{product_id}

Что делает: Удалить товар с фрагмента.

Куда идёт: запись → primary

Пример в Swagger (Try it out → Execute)

region=WEST, product_id=<id>

SQL на фрагменте (то, что реально уходит в PostgreSQL)

```
DELETE FROM products WHERE id = <id> RETURNING id;
```

2.4. Остатки (stock)

GET /api/{region}/stock

Что делает: Список остатков: available и reserved.

Куда идёт: чтение → replica

Пример в Swagger (Try it out → Execute)

region=WEST

SQL на фрагменте (то, что реально уходит в PostgreSQL)

```
SELECT sb.id, w.code AS warehouse, p.sku, p.title,
       sb.qty_available, sb.qty_reserved
FROM stock_balances sb
JOIN warehouses w ON w.id = sb.warehouse_id
JOIN products p ON p.id = sb.product_id
ORDER BY sb.id;
```

POST /api/{region}/stock

Что делает: Завести остаток на складе (qty_reserved стартует с 0).

Куда идёт: запись → primary; склад должен принадлежать region

Пример в Swagger (Try it out → Execute)

region=WEST

Body:

```
{
  "warehouse_id": 1,
  "product_id": 1,
  "qty_available": 25
}
```

SQL на фрагменте (то, что реально уходит в PostgreSQL)

```
INSERT INTO stock_balances (warehouse_id, product_id, qty_available, qty_reserved)
VALUES (1, 1, 25, 0)
RETURNING *;
```

PUT /api/{region}/stock/{stock_id}

Что делает: Поставить новое значение qty_available (ручная правка).

Куда идёт: запись → primary

Пример в Swagger (Try it out → Execute)

region=WEST, stock_id=<id>

Body: {"qty_available": 20}

SQL на фрагменте (то, что реально уходит в PostgreSQL)

```
UPDATE stock_balances SET qty_available = 20 WHERE id = <id> RETURNING *;
```

DELETE /api/{region}/stock/{stock_id}

Что делает: Удалить строку остатка.

Куда идёт: запись → primary

Пример в Swagger (Try it out → Execute)

region=WEST, stock_id=<id>

SQL на фрагменте (то, что реально уходит в PostgreSQL)

```
DELETE FROM stock_balances WHERE id = <id> RETURNING id;
```

2.5. Заявки на отгрузку (orders)

Текущий статус лежит в колонке shipment_orders.status (не отдельная таблица). История — в shipment_events.

GET /api/{region}/orders

Что делает: Список заявок региона.

Куда идёт: чтение → replica

Пример в Swagger (Try it out → Execute)

region=WEST

SQL на фрагменте (то, что реально уходит в PostgreSQL)

```
SELECT o.*, w.code AS warehouse_code
FROM shipment_orders o
JOIN warehouses w ON w.id = o.warehouse_id
WHERE o.region_code = 'WEST'
ORDER BY o.id;
```

POST /api/{region}/orders

Что делает: Создать заявку: status=CREATED + позиция + первая строка журнала. Остатки ещё НЕ трогает.

Куда идёт: запись → primary

Пример в Swagger (Try it out → Execute)

```
region=WEST
Body:
{
  "warehouse_id": 1,
  "product_id": 1,
  "qty": 3
}
```

SQL на фрагменте (то, что реально уходит в PostgreSQL)

```
INSERT INTO shipment_orders (warehouse_id, region_code, status)
VALUES (1, 'WEST', 'CREATED') RETURNING *;
```

```
INSERT INTO shipment_items (order_id, product_id, qty)
VALUES (<order_id>, 1, 3) RETURNING *;
```

```
INSERT INTO shipment_events (order_id, from_status, to_status, note)
VALUES (<order_id>, NULL, 'CREATED', 'order created');
```

POST /api/{region}/orders/{order_id}/status

Что делает: Сменить статус по автомату. При RESERVED/SHIPPED/CANCELLED(from RESERVED) меняет остатки.

Куда идёт: запись → primary + FOR UPDATE на заявке

Пример в Swagger (Try it out → Execute)

```
region=WEST, order_id=<id>
Body:
{
  "to_status": "RESERVED",
  "note": "резерв"
}
```

SQL на фрагменте (то, что реально уходит в PostgreSQL)

```
SELECT * FROM shipment_orders WHERE id = <id> FOR UPDATE;
-- проверка region_code == WEST, иначе 403
-- проверка to_status ∈ TRANSITIONS[current], иначе 400
```

```
-- если RESERVED:
UPDATE stock_balances
SET qty_available = qty_available - <qty>,
    qty_reserved = qty_reserved + <qty>
WHERE warehouse_id = ... AND product_id = ...;
```

```
UPDATE shipment_orders SET status = 'RESERVED', updated_at = NOW() WHERE id = <id>;
INSERT INTO shipment_events (order_id, from_status, to_status, note)
VALUES (<id>, 'CREATED', 'RESERVED', 'резерв');
```

GET /api/{region}/orders/{order_id}/events

Что делает: Журнал переходов одной заявки (аудит).

Куда идёт: чтение → replica; чужой region → 403

Пример в Swagger (Try it out → Execute)

```
region=WEST, order_id=<id>
```

SQL на фрагменте (то, что реально уходит в PostgreSQL)

```
SELECT region_code FROM shipment_orders WHERE id = <id>;
-- если region_code != WEST → 403
SELECT * FROM shipment_events WHERE order_id = <id> ORDER BY id;
```

2.6. Учебный отказ контура

POST /api/cross-region-denied

Что делает: Всегда отвечает 403 — явная демонстрация критерия контура (без записи в БД).

Куда идёт: БД не трогает — сразу HTTPException 403

Пример в Swagger (Try it out → Execute)

```
Query: from_region=WEST, to_region=EAST, order_id=1 → Execute
```

SQL на фрагменте (то, что реально уходит в PostgreSQL)

```
-- SQL нет: отказ на уровне API до обращения к таблицам.
```

2.7. Автомат статусов (шпаргалка)

CREATED → RESERVED | CANCELLED
RESERVED → PICKING | CANCELLED
PICKING → SHIPPED
SHIPPED → DELIVERED
DELIVERED → (конец)
CANCELLED → (конец)

RESERVED: $\text{available} - \text{qty}$, $\text{reserved} + \text{qty}$
CANCELLED из RESERVED: $\text{available} + \text{qty}$, $\text{reserved} - \text{qty}$
SHIPPED: $\text{reserved} - \text{qty}$ (товар ушёл)
CREATED / CANCELLED из CREATED: остатки не трогаем

3. Сценарии для показа в Swagger

Идите по сценариям сверху вниз. После Create сохраняйте id из Response (warehouse_id, product_id, order.id) — они нужны в следующих шагах. Параллельно можно держать psql на WEST primary.

Сценарий А. Подготовка WEST (склад → товар → остаток)

Шаг 1. Показать, что 4 узла ok=true (стенд жив).

Swagger

```
GET /health → Execute
```

Проверка в psql (на primary того же региона)

```
SELECT pg_is_in_recovery(); -- на primary false, на replica true
```

Шаг 2. Создаём склад. В ответе routing.role=primary. Запомните data.id → WH.

Swagger

```
POST /api/{region}/warehouses
region=WEST
{"code":"WH-W-SHOW","name":"Склад показа WEST"}
```

Проверка в psql (на primary того же региона)

```
SELECT * FROM warehouses ORDER BY id DESC LIMIT 3;
```

Шаг 3. Создаём товар только на WEST. Запомните data.id → PR.

Swagger

```
POST /api/{region}/products
region=WEST
{"sku":"SKU-SHOW","title":"Товар для показа"}
```

Проверка в psql (на primary того же региона)

```
SELECT * FROM products WHERE sku='SKU-SHOW';
```

Шаг 4. Заводим остаток 20. reserved должен быть 0.

Swagger

```
POST /api/{region}/stock
region=WEST
{"warehouse_id":WH,"product_id":PR,"qty_available":20}
```

Проверка в psql (на primary того же региона)

```
SELECT * FROM stock_balances WHERE warehouse_id=WH AND product_id=PR;
```

Шаг 5. Чтение: в routing.role обычно replica — подчеркните это преподавателю.

Swagger

```
GET /api/{region}/stock
region=WEST
```

Проверка в psql (на primary того же региона)

```
-- на replica (порт 15433):
SELECT * FROM stock_balances;
```

Сценарий В. Полный успешный путь WEST → DELIVERED

Шаг 1. Заявка CREATED. Остатки пока те же (резерва ещё нет). Запомните order.id → ORD.

Swagger

```
POST /api/{region}/orders
region=WEST
{"warehouse_id":WH,"product_id":PR,"qty":3}
```

Проверка в psql (на primary того же региона)

```
SELECT id, status FROM shipment_orders WHERE id=ORD;
SELECT * FROM shipment_events WHERE order_id=ORD;
```

Шаг 2. Резерв: available 20→17, reserved 0→3.

Swagger

```
POST .../orders/{ORD}/status
{"to_status":"RESERVED","note":"резерв 3"}
```

Проверка в psql (на primary того же региона)

```
SELECT qty_available, qty_reserved FROM stock_balances
WHERE warehouse_id=WH AND product_id=PR;
```

Шаг 3. Статус сменился, остатки не меняются.

Swagger

```
POST .../status → PICKING
{"to_status":"PICKING","note":"сборка"}
```

Проверка в psql (на primary того же региона)

```
SELECT status FROM shipment_orders WHERE id=ORD;
```

Шаг 4. Списание резерва: reserved 3→0 (товар ушёл со склада).

Swagger

```
POST .../status → SHIPPED
{"to_status":"SHIPPED","note":"отгружено"}
```

Проверка в psql (на primary того же региона)

```
SELECT qty_available, qty_reserved FROM stock_balances
WHERE warehouse_id=WH AND product_id=PR;
```

Шаг 5. Финал. Дальше статус менять нельзя.

Swagger

```
POST .../status → DELIVERED
{"to_status":"DELIVERED","note":"доставлено"}
```

Проверка в psql (на primary того же региона)

```
SELECT from_status, to_status FROM shipment_events
WHERE order_id=ORD ORDER BY id;
```

Шаг 6. Показать журнал всей цепочки в Swagger (и при желании в psql).

Swagger

```
GET .../orders/{ORD}/events
region=WEST
```

Проверка в psql (на primary того же региона)

```
SELECT * FROM shipment_events WHERE order_id=ORD ORDER BY id;
```

Сценарий С. Тот же контур на EAST (другой фрагмент)

Шаг 1. Подчеркните: id на EAST свои; данные WEST сюда не «просвечивают».

Swagger

```
POST warehouses / products / stock на region=EAST
(свои code/sku, свои id!)
```

Проверка в psql (на primary того же региона)

```
-- в psql EAST primary (15434):
SELECT code FROM warehouses;
SELECT sku FROM products;
```

Шаг 2. Показать, что автомат тот же, но БД другая.

Swagger

```
POST /api/EAST/orders + статусы до SHIPPED или DELIVERED
```

Проверка в psql (на primary того же региона)

```
SELECT id, region_code, status FROM shipment_orders;
```

Шаг 3. Списки разные: горизонтальная фрагментация по region_code.

Swagger

```
GET /api/WEST/orders и GET /api/EAST/orders
```

Проверка в psql (на primary того же региона)

```
-- WEST primary vs EAST primary – сравнить глазами
```

Сценарий D. Отмена из CREATED (остатки не трогаем)

Шаг 1. Новая заявка в CREATED.

Swagger

```
POST /api/WEST/orders qty=1 → запомнить ORD2
```

Проверка в psql (на primary того же региона)

```
SELECT status FROM shipment_orders WHERE id=ORD2;
```

Шаг 2. Отмена до резерва. available/reserved не меняются.

Swagger

```
POST .../status
{"to_status":"CANCELLED","note":"клиент передумал"}
```

Проверка в psql (на primary того же региона)

```
SELECT qty_available, qty_reserved FROM stock_balances
WHERE warehouse_id=WH AND product_id=PR;
-- сравнить с значениями до отмены – те же
```

Шаг 3. В журнале: NULL→CREATED, затем CREATED→CANCELLED.

Swagger

```
GET .../events ORD2
```

Проверка в psql (на primary того же региона)

```
SELECT from_status, to_status FROM shipment_events WHERE order_id=ORD2;
```

Сценарий E. Отмена из RESERVED (возврат остатка)

Шаг 1. Сначала зарезервировать: available−4, reserved+4.

Swagger

```
POST order qty=4 → status RESERVED
```

Проверка в psql (на primary того же региона)

```
SELECT qty_available, qty_reserved FROM stock_balances  
WHERE warehouse_id=WH AND product_id=PR;
```

Шаг 2. Unreserve: available+4, reserved−4. Товар снова свободен.

Swagger

```
POST .../status  
{ "to_status": "CANCELLED", "note": "сняли резерв" }
```

Проверка в psql (на primary того же региона)

```
SELECT qty_available, qty_reserved FROM stock_balances  
WHERE warehouse_id=WH AND product_id=PR;
```

Сценарий F. CRUD-чтение после записи (primary vs replica)

Шаг 1. В ответе routing.role=primary, host с портом primary.

Swagger

```
POST /api/WEST/products (запись)
```

Проверка в psql (на primary того же региона)

```
--
```

Шаг 2. Обычно role=replica. Если репликация чуть отстаёт — подождите 1–2 с и повторите GET.

Swagger

```
сразу GET /api/WEST/products
```

Проверка в psql (на primary того же региона)

```
-- на replica:  
SELECT * FROM products ORDER BY id DESC LIMIT 5;
```

Сценарий G. Правка остатка без заявки

Шаг 1. Ручная инвентаризация: меняем только available.

Swagger

```
PUT /api/WEST/stock/{stock_id}  
{ "qty_available": 15 }
```

Проверка в psql (на primary того же региона)

```
SELECT id, qty_available, qty_reserved FROM stock_balances WHERE id=<stock_id>;
```

Сценарий H. Список заявок и фильтрация взглядом

Шаг 1. Показать CREATED / RESERVED / DELIVERED / CANCELLED рядом.

Swagger

```
GET /api/WEST/orders
```

Проверка в psql (на primary того же региона)

```
SELECT status, count(*) FROM shipment_orders GROUP BY status;
```

Шаг 2. На EAST своих заявок меньше/другие id — фрагменты независимы.

Swagger

```
GET /api/EAST/orders
```

Проверка в psql (на primary того же региона)

```
-- EAST primary
```

4. Случаи, где «ломается» / не пускает (ожидаемо)

На защите специально покажите 2–3 красных кейса: «система правильно отказала». Это не падение стенда, а демонстрация правил.

✗ Запрещённый переход статуса (400)

Ожидаемый HTTP: 400

Почему ломается / не пускает: Автомат TRANSITIONS не содержит CREATED→SHIPPED. Разрешены только RESERVED и CANCELLED.

Что нажать в Swagger

```
POST /api/WEST/orders/{id}/status
для заявки в CREATED:
{"to_status": "SHIPPED", "note": "прыжок"}
```

Что происходит в БД / SQL

```
SELECT * FROM shipment_orders WHERE id=<id> FOR UPDATE;
-- дальше UPDATE status НЕ выполняется
-- клиент: HTTP 400 «Переход CREATED → SHIPPED запрещён»
```

✗ Отмена из PICKING (400)

Ожидаемый HTTP: 400

Почему ломается / не пускает: После начала сборки отмена запрещена. CANCELLED только из CREATED или RESERVED.

Что нажать в Swagger

```
Довести заявку до PICKING, затем:
{"to_status": "CANCELLED"}
```

Что происходит в БД / SQL

```
-- status остаётся PICKING
-- новая строка в shipment_events НЕ добавляется
```

✗ Недостаточно остатка для резерва (409)

Ожидаемый HTTP: 409

Почему ломается / не пускает: Заявка в CREATED создаётся, но при RESERVED проверка available < qty → конфликт.

Что нажать в Swagger

```
POST order с qty=9999, затем:
{"to_status": "RESERVED"}
```

Что происходит в БД / SQL

```
SELECT qty_available FROM stock_balances ... FOR UPDATE;
-- if available < qty → HTTP 409 «Недостаточно остатка для резерва»
-- status заявки остаётся CREATED; reserved не растёт
```

✗ Критерий контура: чужой регион в URL (403)

Ожидаемый HTTP: 403 (или 404 на пустом фрагменте)

Почему ломается / не пускает: Строка заявки WEST существует только на фрагменте WEST. Через EAST её менять нельзя.

Что нажать в Swagger

```
Взять order_id заявки WEST.
POST /api/EAST/orders/{WEST_id}/status
{"to_status": "CANCELLED"}
```

Что происходит в БД / SQL

```
-- на EAST: SELECT * FROM shipment_orders WHERE id=<WEST_id>;
-- часто пусто → 404 «не найдена на этом фрагменте»
-- либо (если id случайно совпал) проверка region_code → 403
Альтернатива без БД: POST /api/cross-region-denied?from_region=WEST&to_region=EAST&order_id=1
```

✗ Отказ primary: запись невозможна (503)

Ожидаемый HTTP: 503 на записи; 200 на чтении с replica

Почему ломается / не пускает: Запись идёт только на primary. Узел лежит → клиент не может записать.

Что нажать в Swagger

В терминале: `docker stop rdb-west-primary`
Затем в Swagger: `POST /api/WEST/orders ...`

Что происходит в БД / SQL

-- `connect(WEST, primary)` → ошибка сети
-- HTTP 503 «Узел недоступен: region=WEST, role=primary»

Сразу покажите контраст:

`GET /api/WEST/stock` → 200 с replica (чтение живо)
`GET /api/EAST/stock` → 200 (другой фрагмент не затронут)

Восстановление:

`docker start rdb-west-primary`
подождать healthy, снова `POST orders`

✗ Переход из DELIVERED назад (400)

Ожидаемый HTTP: 400

Почему ломается / не пускает: DELIVERED — терминальный статус, множество переходов пустое.

Что нажать в Swagger

Для заявки уже в DELIVERED:
`{"to_status": "PICKING"}`

Что происходит в БД / SQL

-- `UPDATE` не выполняется; `events` без новой строки

4.1. Мини-скрипт отказа узла (терминал рядом со Swagger)

```
docker stop rdb-west-primary           # «убили» primary WEST
# Swagger: POST /api/WEST/orders       → отказ (красно)
# Swagger: GET /api/WEST/stock         → OK с replica
# Swagger: GET /api/EAST/warehouses    → OK (EAST жив)
docker start rdb-west-primary          # вернули
docker exec rdb-west-primary pg_isready -U warehouse -d warehouse
```

5. Короткий чек-лист показа (5-7 минут)

- 1) `/health` — 4 узла зелёные.
- 2) `POST warehouse + product + stock` на WEST; `GET stock` → `role=replica`.
- 3) `POST order` → `RESERVED` (показать `available/reserved` в `psql`) → ... → `DELIVERED` → `events`.
- 4) На EAST создать свой склад/товар — «это другой фрагмент».
- 5) Красный: `CREATED` → `SHIPPED` → 400.
- 6) Красный: статус WEST-заявки через EAST → 403 (или `cross-region-denied`).
- 7) По желанию: `docker stop west-primary` → запись падает, чтение с replica живо → `start`.

6. Подъём стенда одной командой

Чтобы сразу показать преподавателю живой стенд (Docker + схема + API + демо):

```
cd ~/Рабочий\ стол/РБД/Индивидуальный_проект && bash scripts/run_everything.sh
```

потом браузер:
`http://127.0.0.1:8000/docs`

6.1. Что лежит в scripts/

Скрипт	Назначение
<code>run_everything.sh</code>	Всё сразу: <code>compose</code> , схема, <code>uvicorn</code> , демо, скрины
<code>bootstrap_schema.sh / .py</code>	Накатить схему и <code>seed</code> на WEST/EAST primary
<code>demo.sh</code>	Прогон API с пояснениями (нужен <code>uvicorn</code>)
<code>capture_screenshots.py</code>	PNG Swagger для отчёта
<code>make_dump.sh</code>	<code>pg_dump</code> → папка <code>dump/</code> к сдаче

Логика API — в `client/app/main.py` (не в `scripts`). Скрипты только поднимают окружение и дергают HTTP.

6.2. По шагам (если одна команда не нужна)

```
docker compose up -d
```

```
./scripts/bootstrap_schema.sh
source .venv/bin/activate
uvicorn client.app.main:app --port 8000
./scripts/demo.sh
```

Файл сгенерирован скриптом docs/build_swagger_guide_pdf.py · пересборка: python docs/build_swagger_guide_pdf.py